

Deliverable 4 Technical Report and NFRs and Architectural Design & Justifications

1. Introduction

Our School Management System is a centralized digital platform designed to automate and streamline academic administration, including student record management, grade distribution, and real-time attendance tracking. By moving from a standalone application to a structured architectural model, we ensure that the system can handle concurrent access from students and teachers while maintaining data integrity. This deliverable documents the transition from a creational pattern-focused design to a robust, layered architecture that defines how these components interact and deploy in a real-world environment.

2. Architectural Design & Justification

We have adopted a **Layered Architecture** integrated with the **Model-View-Controller (MVC)** architectural style.

Justification for Choice

- **Suitability for Domain:** A school system requires strict data validation and clear user roles. The **Layered Architecture** allows us to separate the user interface (Presentation Layer) from the database logic (Data Access Layer), ensuring that students cannot bypass the UI to modify grades.
- **Support for Requirements:** The **MVC Pattern** allows us to manage the complex "Manage User Accounts" and "Grade Viewer" scenarios independently. We can update the teacher's dashboard (the View) without needing to rewrite the grade-calculation logic (the Model).
- **Pattern Complementarity:** These patterns work in tandem: the Layered Architecture defines the physical organization of our Java packages, while MVC handles the dynamic data-flow within those packages to keep the system decoupled and organized.

Layered Architecture Suitability

- **Encapsulation of Logic:** We chose a 3-tier Layered Architecture to enforce a strict boundary between the **Presentation Layer** (UI), **Business Logic Layer** (Design Patterns), and **Data Access Layer** (Database).

- **Security Through Isolation:** This structure ensures that the Data Access Layer is never exposed directly to the user. For example, when a student requests to "View Grades," the request must pass through the Business Logic where our **Singleton** GradeViewerService validates the session before fetching data.

MVC Pattern Support

- **Decoupling the UI:** The **Model-View-Controller** pattern allows us to maintain the "School Management System" efficiently. If we decide to upgrade the student dashboard from a desktop UI to a web-based interface, we only need to replace the **View**; the **Model** (our data structures) remains untouched.
- **Handling User Roles:** Because our system serves distinct roles (Students, Teachers, Admins), MVC allows the **Controller** to route users to the correct **View** based on their permissions, which we managed using the **Abstract Factory Pattern** in the last deliverable.

3. Non-Functional Requirements (NFRs)

Our architectural decisions were specifically chosen to support the following quality attributes:

- **Security:** By using the **Singleton Pattern** for the GradeViewerService, we create a single, controlled gateway for all database interactions. This prevents unauthorized or duplicate connections that could lead to data leaks or record corruption.
- **Performance:** The architecture minimizes resource usage by reusing service objects for attendance and grade viewing. This prevents the system from slowing down during high-traffic events, such as when entire classrooms view exam results simultaneously.
- **Availability:** Through the use of the **Strategy Pattern** for our notification system, the core application remains available even if external dependencies (like an SMS gateway) fail. The system simply swaps the delivery method without crashing the entire session.

Availability Detailed Explanation

- **Architectural Support:** Our design utilizes the **Strategy Pattern** for the "Dynamic Notification System".
- **Detailed Explanation:** By decoupling the notification delivery from the main application flow, we ensure high **Availability**. If the external **Third-Party SMS Gateway** is down, the system does not hang or crash. Instead, it can fallback to the "Dashboard Announcement" strategy, ensuring the message is still delivered internally.

- **Maintainability:** We utilized **Template Methods** for the "Course Creation" and "Attendance Marking" workflows. This makes the system easy to maintain; if the school changes its attendance policy, we only need to update the specific subclass without touching the core system logic.

Maintainability & The Open/Closed Principle

- **Architectural Support:** The system heavily relies on the **Template Method Pattern** for "Course Creation" and "Attendance Marking".
- **Detailed Explanation:** This architecture supports the **Open/Closed Principle**—the system is "closed" to modification of the core workflow but "open" to extensions. We can add a "Biometric Attendance" subclass next week without changing a single line of the original AttendanceProcess base class, making the system incredibly easy to maintain as technology evolves.
- **Scalability:** The **Abstract Factory Pattern** allows us to scale the system across different platforms (Web and Mobile) by creating UI families independently. Adding a new platform for "Parent Viewers" in the future will not require restructuring existing code.

4. Process View: Emergency Alert Workflow Logic

For the Activity and Sequence diagrams, we focused on the Emergency Alert Workflow to demonstrate runtime interaction between components:

1. Request Initiation: An Administrator provides a message string to the CommunicationModule.
2. Structural Decoration: The system creates a DashboardAnnouncement object and wraps it in an EmailDecorator and SMSDecorator.
3. Behavioral Execution: When the send() method is called, it triggers a chain reaction: the basic dashboard post is made first, followed by the email alert, and finally the SMS dispatch.
4. Strategic Adaptation: The SMSDecorator uses a specific NotificationDeliveryStrategy to interface with the SMSSGatewayAdapter, which handles the translation of data for the external legacy hardware.

In our **School Management System**, the **Emergency Alert Workflow** is the logical process that handles urgent communication across multiple platforms simultaneously. It primarily utilizes the **Decorator**, **Strategy**, and **Adapter** patterns to ensure that a single announcement reaches everyone through various channels.

Here is the step-by-step breakdown of how this process works in your software architecture:

1. Initiation & Logic Hub

The Trigger: An Administrator enters an urgent message (e.g., "School closed due to weather") into the CommunicationModule.

Strategic Selection: The system uses the **Strategy Pattern** to act as a "universal remote," selecting the best NotificationDeliveryStrategy (Email, SMS, or In-App) based on the alert's priority.

2. Core Announcement & Decoration

The Base Component: The message is first sent to the DashboardAnnouncement class, which posts the alert directly to the internal school dashboard.

Dynamic Wrapping: Using the **Decorator Pattern**, the system "wraps" this basic announcement with additional layers of responsibility without changing the original code.

Multi-Channel Execution: The EmailDecorator adds the functionality to send email alerts to all subscribed students and teachers , while the SMSDecorator adds the capability to send emergency texts to parent contacts.

3. External Gateway Adaptation

The Problem: The system's internal notification interface is often incompatible with external, third-party SMS libraries.

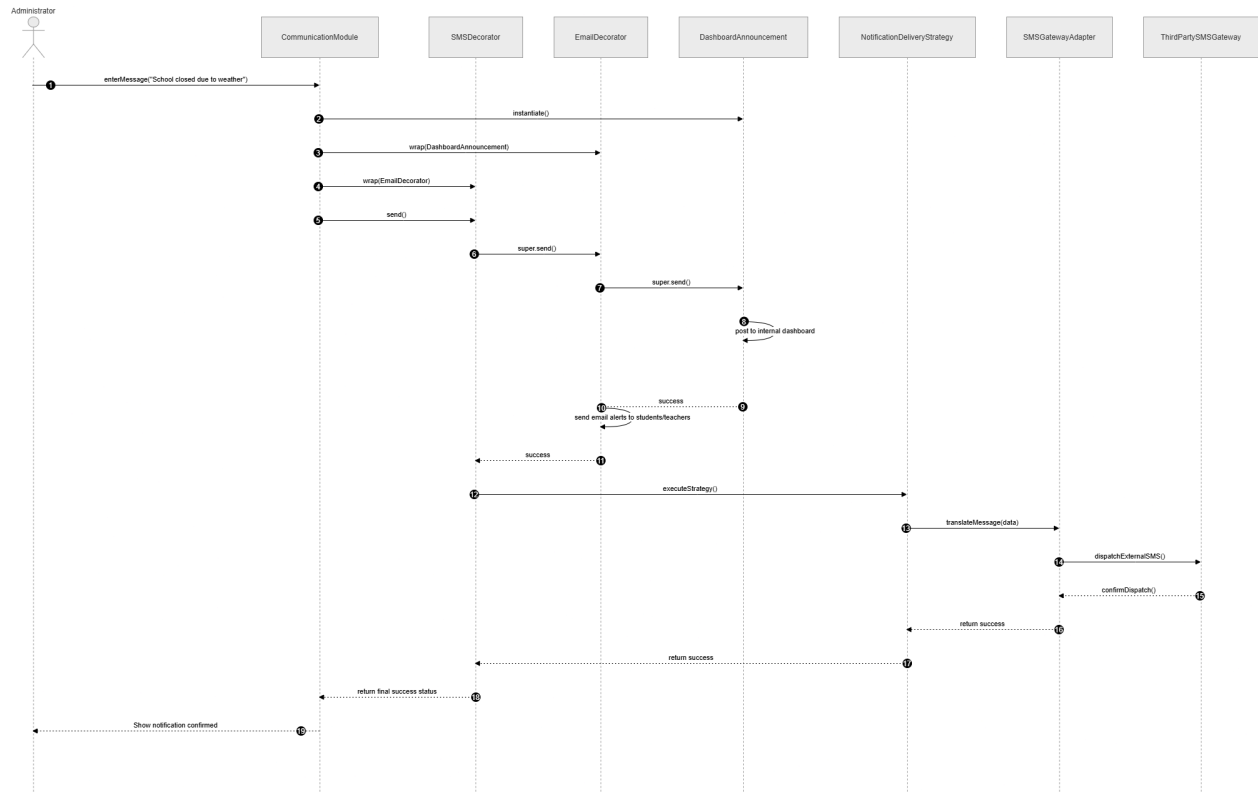
The Solution: The SMSGatewayAdapter acts as a bridge, translating the school system's message into a format that the ThirdPartySMSGateway can understand and dispatch.

4. Final Confirmation

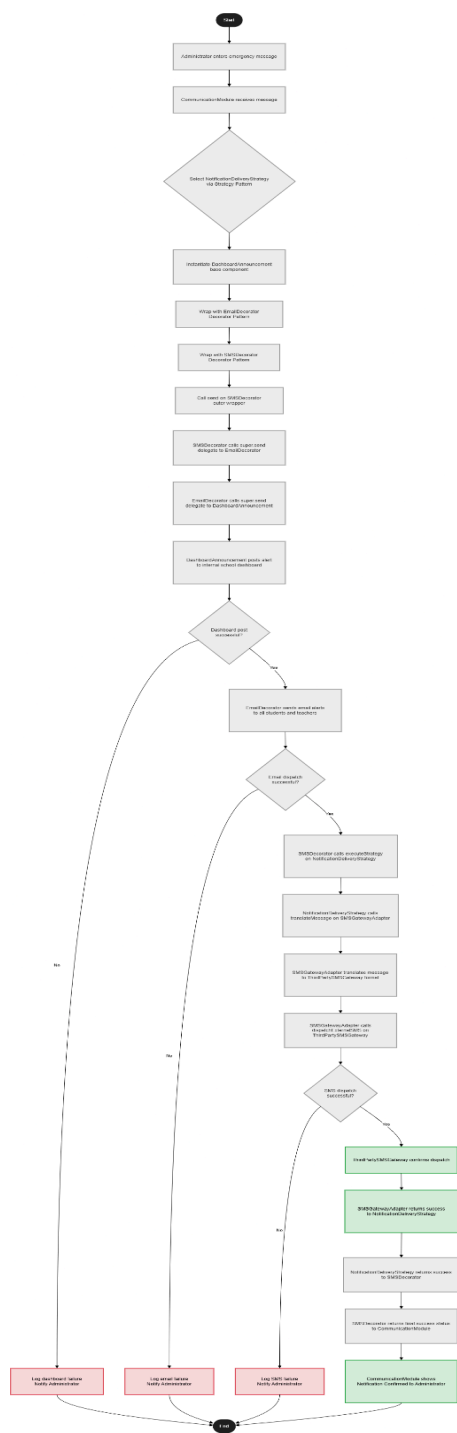
Verification: Once the external gateways confirm the dispatch, the NotificationService returns a success status.

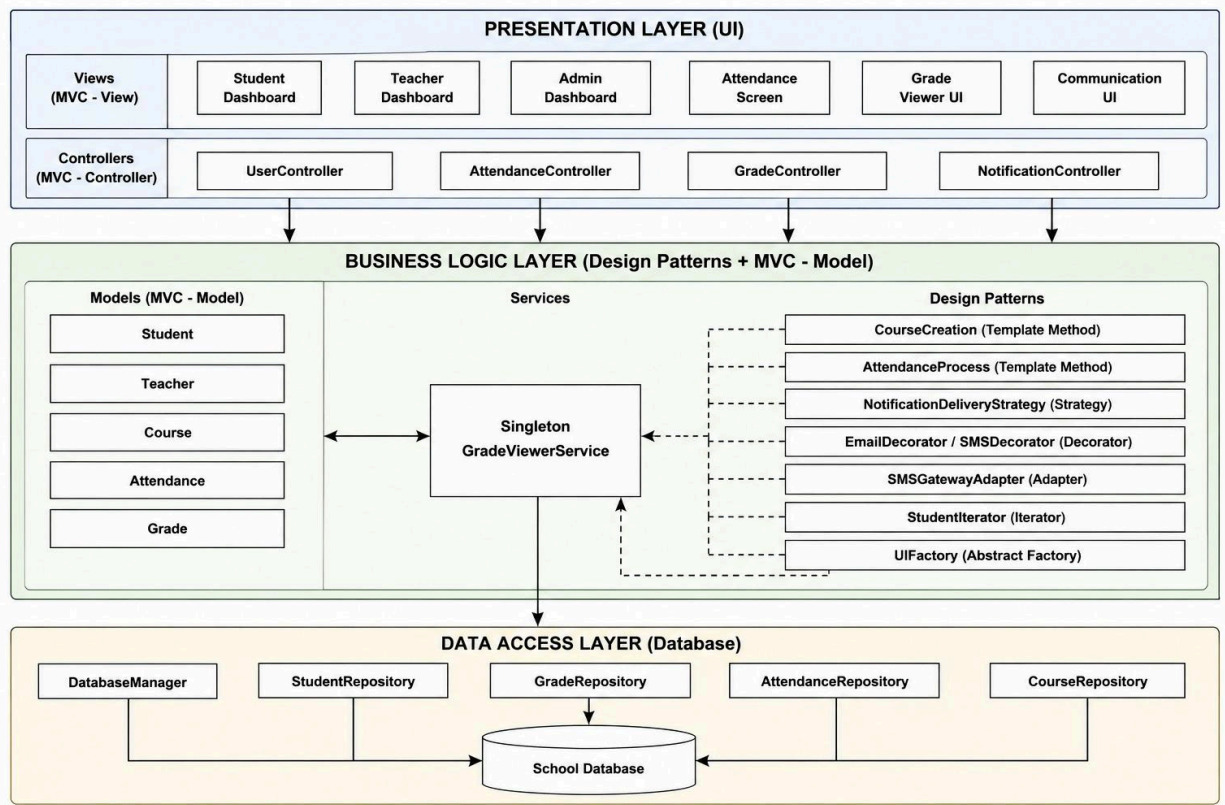
System Stability: Because of this decoupled architecture, if one channel (like the SMS provider) fails, the rest of the system remains available and the dashboard post still goes through.

Process View Sequence Diagram:



Process View Activity Diagram:





Physical View Deployment Diagram:

